

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

Звіт
з Лабораторної роботи №2
з дисципліни: «Основи програмної інженерії»
на тему:
«Моделювання програмної системи засобами UML
(Unified Modeling Language)»

Виконала: ст. гр. ПЗПІ-25-3
Гиренко В. О.

Прийняв: Гребенюк В. О.

Харків 2026

МЕТА РОБОТИ

Набуття практичних навичок моделювання програмних систем із використанням уніфікованої мови моделювання UML (Unified Modeling Language). Оволодіння методами побудови діаграм прецедентів (Use Case Diagram), діаграм класів (Class Diagram) та діаграм послідовності (Sequence Diagram). Формування вміння забезпечувати трасовність (traceability) від функціональних вимог до моделей проектування.

ХІД ВИКОНАННЯ РОБОТИ

1.1 Обрання предметної області

HookCraft (з яким працювали на пз 1 та 2) — це вебсервіс на базі штучного інтелекту, призначений для швидкого створення ефективних «гачків» (hooks) та перших речень для постів, відео й рекламних оголошень, що мають на меті зачепити увагу аудиторії.

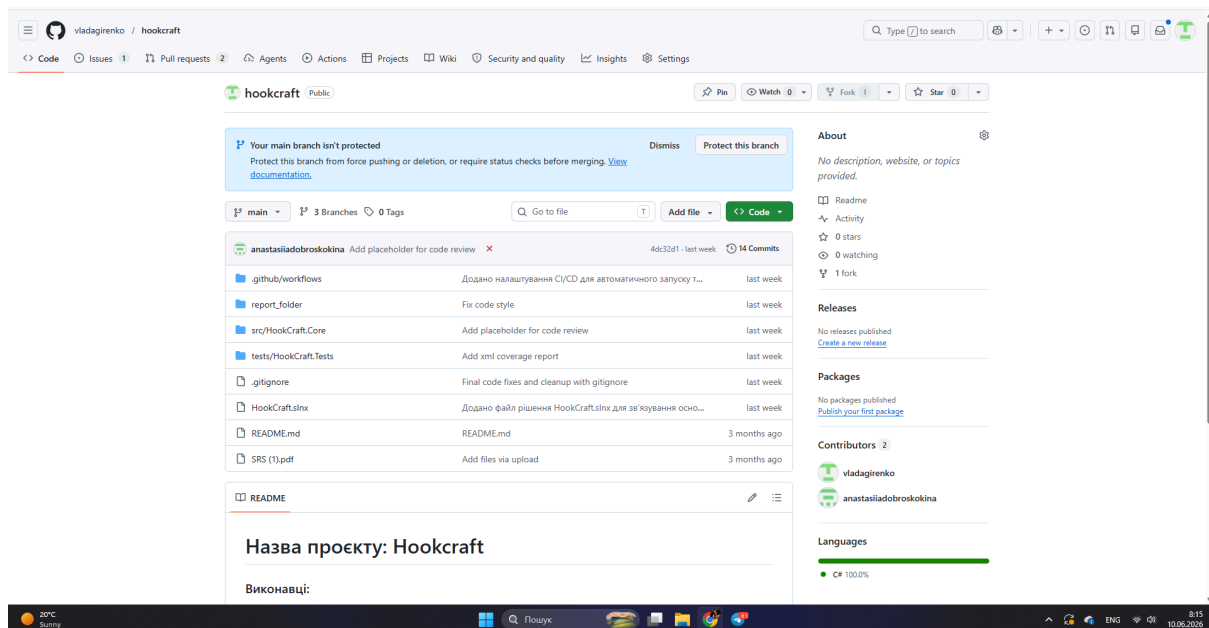


Рисунок 1.1 – Скріншот репозиторію Hookcraft

1.2 Обрання функціональних вимог

Таблиця 1.1 – Функціональні вимоги

Ідентифікатор	Пріоритет	Опис функціональної вимоги
FR-01	[Must]	Система повинна дозволяти користувачу вводити тему або опис контенту.

FR-02	[Must]	Система повинна генерувати кілька варіантів текстових «гачків» за допомогою AI.
FR-03	[Must]	Користувач повинен мати можливість обрати стиль тексту (наприклад: інтригуючий, гумористичний).
FR-04	[Must]	Система повинна відображати список згенерованих варіантів тексту.
FR-05	[Must]	Користувач може скопіювати обраний текст у буфер обміну.
FR-06	[Should]	Зареєстрований користувач може переглядати історію своїх генерацій.
FR-07	[Should]	Система повинна підтримувати реєстрацію користувача через email або Google.

1.3 Побудова діаграми прецедентів

Актори:

- Гість (незареєстрований користувач).
- Зареєстрований користувач.
- Адміністратор.

Прецеденти:

- Ввести тему (Enter Topic)
- Обрати стиль (Choose Style)
- Згенерувати гачки (Generate Hooks)
- Переглянути результати (View Results)
- Скопіювати текст (Copy Text)
- Переглянути історію (View History)
- Зареєструватись / увійти (Register / Login)
- Керувати лімітами (Manage Limits) - для адміністратора

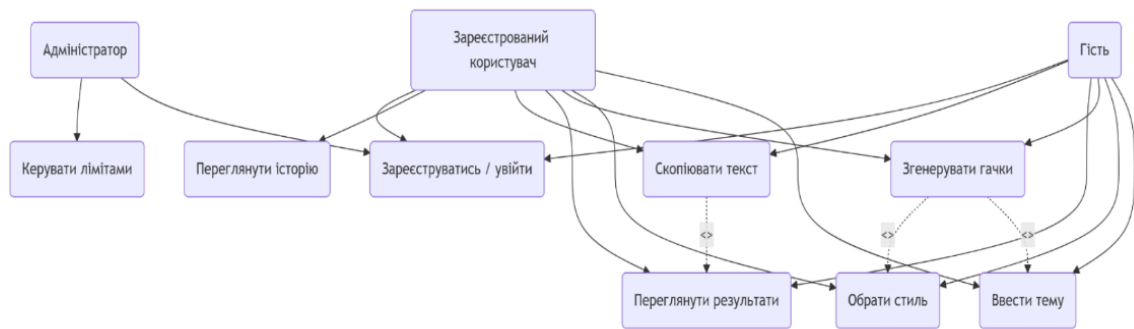


Рисунок 1.2 – Діаграма прецедентів (виконав Крутько Дмитро)

1.4 Побудова діаграми класів

Класи:

- User (абстрактний або базовий) – атрибути: id, email, role, registrationDate; методи: authenticate(), getHistory()
- Guest (наслідує User) – додаткові атрибути: remainingGenerations; методи: useTrial()
- RegisteredUser (наслідує User) – атрибути: favoriteHooks, historyList; методи: saveToFavorites(), viewHistory()
- Admin (наслідує User) – методи: manageUsers(), updateLimits()
- GenerationRequest – атрибути: topic, style, timestamp, userId; методи: submit(), getResult()
- Hook – атрибути: id, text, style, isFavorite, createdAt; методи: copyToClipboard(), getText()
- History (композиція з RegisteredUser) – атрибути: listOfHooks; методи: addHook(), clearHistory()
- AuthService – методи: login(email, password), register(email, password), logout()

Зв'язки:

- Наслідування: Guest, RegisteredUser, Admin $\rightarrow$ User.
- Асоціація: User 1 – * GenerationRequest.
- Композиція: RegisteredUser 1 – * History (без користувача історія не існує).
- Агрегація: History * – * Hook (гачки можуть існувати окремо).
- Залежність: GenerationRequest $\dots$ AuthService.

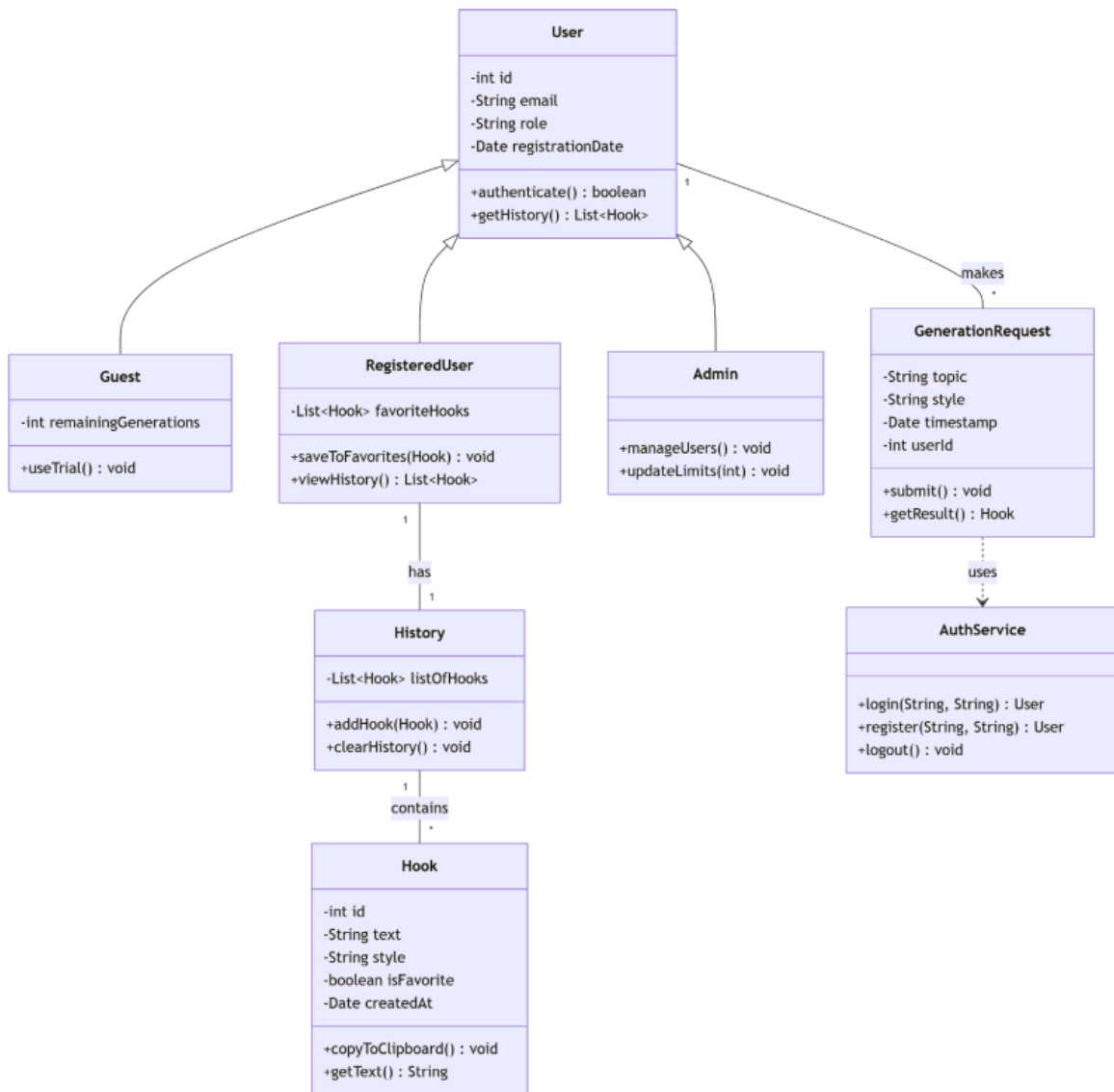


Рисунок 1.3 – Діаграма класів (виконала Анастасія Доброскоккіна)

1.5 Побудова діаграми послідовності

Обраний сценарій: "Генерація гачків зареєстрованим користувачем" (включає FR-01, FR-02, FR-03, FR-04).

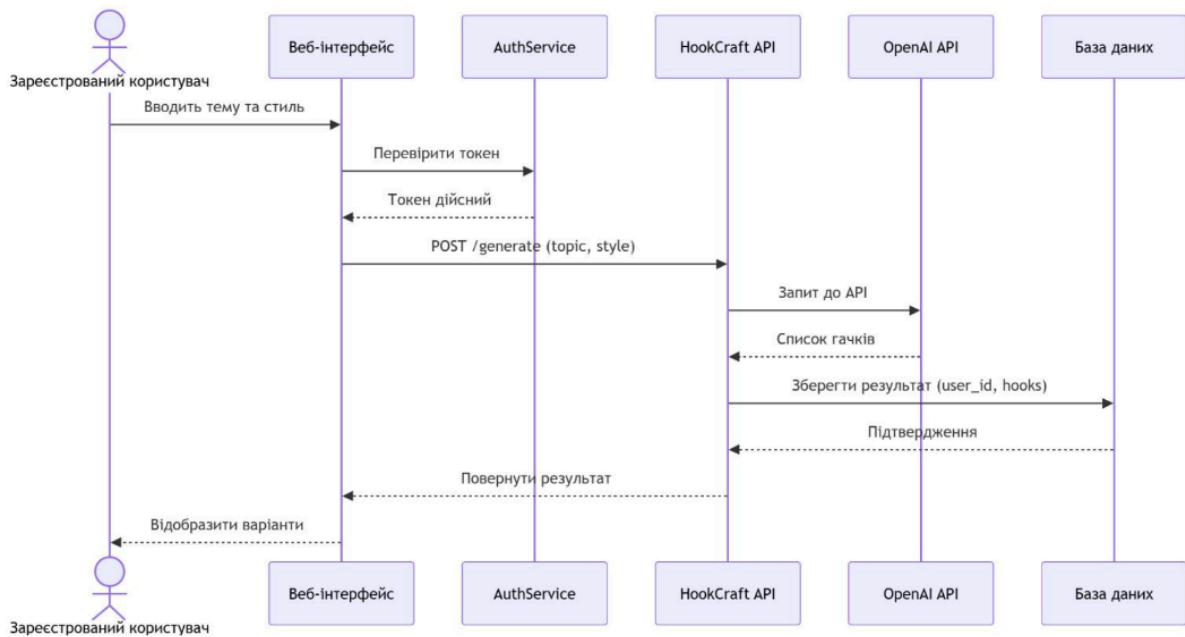


Рисунок 1.4 – Діаграма послідовності (виконала Гиренко Владислава)

1.6 Складання матриці трасовності

Побудували матрицю трасовності за зразком: для кожної функціональної вимоги вказала відповідний прецедент, задіяні класи та діаграму послідовності.

Прецедент UC-08 «Керувати лімітами» призначений для Адміністратора і не відповідає жодній з 7 обраних функціональних вимог (це внутрішня функція системи, яка не є прямою вимогою користувача). Тому в матриці він відсутній.

Таблиця 1.2 – Матриця трасовності

Вимога	Прецедент	Класи	Діаграма послідовності
FR-01	UC-01 "Ввести тему"	User, GenerationRequest	SD-01 (фрагмент)
FR-02	UC-03 "Згенерувати гачки"	GenerationRequest, Hook, AuthService	SD-01
FR-03	UC-02 "Обрати стиль"	GenerationRequest	SD-01 (фрагмент)
FR-04	UC-04 "Переглянути результати"	Hook, UI	SD-01 (фрагмент)

FR-05	UC-05 "Скопіювати текст"	Hook	—
FR-06	UC-06 "Переглянути історію"	RegisteredUser, History, Hook	—
FR-07	UC-07 "Зареєструватись / увійти"	AuthService, User	—

ВИСНОВКИ

У ході лабораторної роботи ми структурували сім ключових функціональних вимог системи від FR-01 до FR-07. Виділили трьох акторів та вісім прецедентів взаємодії, що повністю описують усі базові сценарії роботи сервісу. На основі аналізу предметної області було спроектовано структуру з восьми класів із точним налаштуванням зв'язків наслідування, асоціації, композиції та агрегації. Побудували діаграму послідовності (включає FR-01, FR-02, FR-03, FR-04) та матрицю трасовності за зразком: для кожної функціональної вимоги вказала відповідний прецедент, задіяні класи та діаграму послідовності.